

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COMMON COURSE ASSIGNMENT REPORT

Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System using Java

Particular	Details
Department:	AIML
Programme:	BTECH
Course Code & Course Name	CSA0911-PROGRAMMING IN JAVA
Academic Year / Batch	2024
Faculty Name	R. THALAPATHI RAJASEKARAN
Assignment Title	Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System using Java
Maximum Marks	100
Course Outcome(s) – CO	CO1: Apply OOP principles in Java for designing patient, doctor, appointment, medicine, inventory and notification classes. CO2: Implement Java Collections, Generics and Iterators for managing patients, appointments, doctors, medicines and waitlists. CO3: Develop robust Java applications using custom exception handling for invalid patient IDs, duplicate appointments, invalid inputs and insufficient medicine stock. CO4: Design concurrent multithreaded systems using thread synchronization and inter-thread communication for appointment booking, inventory updates and notification processing.
Bloom's Taxonomy Level	Level 3 (Apply), Level 4 (Analyze), Level 5 (Evaluate), Level 6 (Create)
SDG Mapping (SDG 1-17)	SDG 3: Improves healthcare service management through efficient patient appointments, pharmacy inventory tracking, reminders, and alerts. SDG 9: Uses Java OOP, collections, GUI, multithreading, synchronization, and automated notifications to build an efficient digital healthcare management system. SDG 10: Supports fair appointment access through waitlisting

	and patient-category considerations, helping improve accessibility for different patient groups.
Industry / Societal Relevance	The system improves healthcare services through efficient appointment scheduling, pharmacy inventory management, and automated alerts.

Team Members

S.No	Name	Register No.	Role in Team
1	V.Abhinaya	192421208	OOP & Patient
2	S.Dharshini	192421040	Collections & Pharmacy
3	V.Rakshana	192425137	GUI & Integration

1. Problem Statement and Problem Formulation

1.1 Assignment Problem

Design, implement, and evaluate a menu-driven Java application to manage a Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System. Patients book appointments and are prescribed medicines; medicine stock is a limited, shared resource that must be tracked accurately; and appointment-fulfilment activities must be handled reliably even when several actions happen at once.

1.2 Detailed Problem Statement

The application automates the following hospital front-office and pharmacy workflow:

- Registering patients and doctors/medicines.
- Displaying available appointment slots and medicine stock.
- Booking and cancelling appointments.
- Searching and updating patient / medicine records.
- Maintaining a waitlist for fully booked doctors, with automatic promotion on cancellation.
- Generating patient-visit and pharmacy-inventory reports.
- Sending appointment-reminder and low-stock notifications.

Core entities - Patient, Doctor/Appointment, Medicine, Inventory, and Notification - are modelled as encapsulated classes with private data members and cohesive member methods . Patient categories and medicine types are organised through inheritance and polymorphism so that consultation-fee waivers, dosage rules, and notification behaviour adapt at run time. The Java Collection Framework (ArrayList, HashSet, HashMap, Hashtable) is used with generics and Iterator/ListIterator for storing, retrieving, updating, and traversing records . Built-in and user-defined exceptions handle invalid patient IDs, duplicate appointments, out-of-stock medicines, and invalid input. Multithreading simulates concurrent appointment booking and

notification dispatch using thread priorities, synchronization, and inter-thread communication (wait/notify) to safely update shared appointment-slot and inventory data .

1.3 Problem Formulation Summary

Aspect	Formulation
What is the problem?	A single shared hospital front-desk + pharmacy system is needed where registration, booking, dispensing and alerting must stay correct even under concurrent access, with no data loss or crashes on bad input.
Expected outcomes	A working console application (menu-driven) that registers patients/doctors/medicines, books/cancels appointments with waitlisting, dispenses medicine against live stock, raises reminder and low-stock alerts, and prints patient-visit and inventory reports.
Information	Patient categories (General, Senior Citizen, Insured), doctor specialisations and slot counts, medicine types (Tablet, Syrup, Injectable) with unit price, stock quantity and reorder threshold.
Assumptions	Single JVM process (no persistence to a database is required for this assignment); one appointment slot = one time unit per doctor per day; consultation fee is computed per visit, not billed separately; notification 'sending' is simulated via console/log output rather than SMS/email gateways.
Constraints	Must use core OOP (encapsulation, inheritance, polymorphism); must use ArrayList, Set, HashMap and Hashtable; must use Iterator/ListIterator; must use both built-in and custom checked exceptions; must use real threads with priorities, synchronization and inter-thread communication; 2-day submission window; team of 3.

2. Objective and Expected Outcomes

- Model the hospital domain using well-encapsulated classes with clear single-responsibility boundaries (Person -> Patient/Doctor, Medicine, Appointment, Notification, Inventory, HospitalService).
- Demonstrate inheritance and run-time polymorphism through patient-fee categories, medicine dosage types, and notification types.
- Apply the Collection Framework (ArrayList, HashSet, HashMap, Hashtable) with generics and explicit Iterator/ListIterator traversal for search, update and removal.
- Handle invalid patient IDs, duplicate appointments, out-of-stock medicine and bad input using custom checked exceptions without ever terminating the program.
- Simulate concurrent booking and notification dispatch safely using synchronized methods, thread priorities, and a hand-built wait()/notifyAll() producer-consumer queue.
- Produce a clean, menu-driven console interface plus patient-visit and inventory-utilisation reports.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- FR1: Register patients under General / Senior Citizen / Insured categories and doctors with a specialisation and slot count.
- FR2: Book an appointment for a registered patient with a registered doctor; reject duplicates; waitlist when full.
- FR3: Cancel an appointment, freeing the slot and auto-promoting the next waitlisted patient, if any.
- FR4: Search a patient by ID and update editable fields (e.g. contact number).
- FR5: Dispense a medicine against a patient's prescription, deducting live stock.
- FR6: Raise a low-stock alert automatically when a medicine's quantity falls to or below its reorder threshold.
- FR7: Raise an appointment-reminder alert automatically when a booking succeeds.
- FR8: Generate a patient-visit report (all appointments with status) and a pharmacy inventory-utilisation report (stock + active low-stock alerts).
- FR9: Demonstrate concurrent booking (multiple threads competing for the same doctor's slots) with a background thread dispatching queued notifications.

3.2 Non-Functional Requirements

- Robustness: no user input or business-rule violation should crash the program; every exception path is caught and reported.
- Thread-safety: shared mutable state (doctor slot counts, medicine stock, the notification queue) must never be corrupted by concurrent access.
- Usability: a clear numbered console menu with input validation.
- Maintainability: package-per-concern layout, encapsulated fields, small cohesive methods.

3.3 Constraints

- Language/Platform: Java (JDK 17+), console-based, no external frameworks required.
- Must satisfy CO1 (OOP), CO2 (Collections), CO3 (Exceptions + Multithreading) as mapped in the rubric.
- 2-day submission window; deliverables are Pseudocode, Implementation & Results, and a GitHub upload.

3.4 Assumptions

- No persistent database is required; all data lives in in-memory collections for the duration of a run.
- 'Sending' a notification is simulated by formatting and printing/logging the message rather than integrating a real SMS/email gateway.
- One doctor slot represents one bookable appointment; slot capacity resets are out of scope for this assignment.

4. Application of Relevant Course Knowledge / Concepts

4.1 Object-Oriented Principles

- Encapsulation & data hiding: every entity (Person, Patient, Doctor, Medicine, Appointment, Notification) keeps its fields private with controlled getters/setters; mutating operations that touch shared state (reserveSlot(), deduct()) are the only entry points, so external code can never corrupt an object's invariants directly.
- Inheritance: Patient and Doctor extend an abstract Person; GeneralPatient, SeniorCitizenPatient and InsuredPatient extend Patient; TabletMedicine, SyrupMedicine and InjectableMedicine extend an abstract Medicine; ReminderNotification and LowStockNotification extend an abstract Notification.
- Polymorphism: calculateConsultationFee(), getDosageInstructions() and send() are declared abstract in the parent and overridden differently in each subclass, so calling code invokes them through the parent type and gets the correct behaviour at run time (dynamic dispatch).

4.2 Collection Framework, Generics & Iterators

Structure	Used for	Why this structure
ArrayList<Patient>	Ordered patient registry, appointment list	Fast indexed/sequential access; insertion order matters for reports.
HashMap<String,Doctor>	Doctor lookup by ID	O(1) average lookup when booking by doctor ID; no ordering needed.
HashSet<String>	Registered patient/doctor IDs	O(1) duplicate-ID detection during registration.
Hashtable<String,Medicine>	Pharmacy inventory	Legacy synchronized Map: every get/put is internally thread-safe, matching the multithreaded stock-update requirement.

Iterator / ListIterator	Patient search, appointment traversal & cancellation, low-stock scan	Explicit, generic-safe traversal that also supports mid-loop mutation (via ListIterator) when cancelling an appointment.
-------------------------	--	--

4.3 Exception Handling

- Custom checked exceptions: InvalidPatientIdException, DuplicateAppointmentException, OutOfStockException, DoctorUnavailableException, InvalidInputException - each models one specific business rule violation.
- Built-in exceptions: NumberFormatException is caught around every numeric menu input.
- try/catch is centralised in the Main menu loop so a single bad action reports an error and returns to the menu instead of terminating the JVM.

4.4 Multithreading, Synchronization & Inter-thread Communication

- Thread priorities: BookingSimulator threads run at NORM_PRIORITY+1 (booking is time-critical); NotificationDispatcher runs at NORM_PRIORITY-1 (alerting can lag slightly).
- Synchronization: Doctor.reserveSlot()/releaseSlot(), Medicine.deduct()/restock(), and HospitalService.bookAppointment()/cancelAppointment() are all synchronized so two threads racing for the same resource cannot both succeed or corrupt a count.
- Inter-thread communication: NotificationQueue is a hand-built producer-consumer queue using wait()/notifyAll() - producers (booking/dispensing flows) enqueue and notify; the consumer thread waits inside a synchronized block until woken, demonstrating the classic wait/notify pattern explicitly rather than hiding it behind a library BlockingQueue.

5. Design / Proposed Solution / Methodology

5.1 Package Layout

```

hospital/
  Person.java           (abstract)
  Patient.java          (abstract, extends Person)
  GeneralPatient.java
  SeniorCitizenPatient.java
  InsuredPatient.java
  Doctor.java           (extends Person)
  Medicine.java         (abstract)
  TabletMedicine.java
  SyrupMedicine.java
  InjectableMedicine.java
  Appointment.java

```

```

Inventory.java
Notification.java          (abstract)
ReminderNotification.java
LowStockNotification.java
NotificationQueue.java     (wait/notify producer-consumer)
BookingSimulator.java      (Thread, producer)
NotificationDispatcher.java (Thread, consumer)
HospitalService.java       (core service / collections)
ReportGenerator.java
Main.java                  (menu-driven entry point)
exceptions/
  InvalidPatientIdException.java
  DuplicateAppointmentException.java
  OutOfStockException.java
  DoctorUnavailableException.java
  InvalidInputException.java

```

5.2 Class Relationships

- Person <|-- Patient <|-- {GeneralPatient, SeniorCitizenPatient, InsuredPatient}
- Person <|-- Doctor (Doctor 1 --- * Appointment; Doctor 1 --- * waitlisted Patient)
- Medicine <|-- {TabletMedicine, SyrupMedicine, InjectableMedicine}
- Notification <|-- {ReminderNotification, LowStockNotification}
- HospitalService --- ArrayList<Patient>, HashMap<String,Doctor>, HashSet<String>, ArrayList<Appointment>, Inventory (Hashtable<String,Medicine>)
- BookingSimulator (Thread) --> HospitalService.bookAppointment() [synchronized] --> NotificationQueue.enqueue()
- NotificationDispatcher (Thread) --> NotificationQueue.dequeue() [wait()] --> Notification.send()

5.3 Key Design Decisions

- Abstract base classes (Person, Patient, Medicine, Notification) capture shared state/behaviour once and force each subclass to supply only what differs - keeping the fee/dosage/notification rules open for extension without touching existing code (Open/Closed Principle).
- Slot reservation and stock deduction are pushed down into the owning object's own synchronized method (Doctor.reserveSlot(), Medicine.deduct()) rather than synchronizing the whole HospitalService, so unrelated doctors/medicines are not blocked by each other's locks - a finer-grained locking strategy.
- The notification queue is built by hand with wait()/notifyAll() (rather than java.util.concurrent.LinkedBlockingQueue) specifically to make the inter-thread communication mechanism visible and assessable.

- Waitlisting uses a Queue<Patient> (LinkedList) per doctor so promotion on cancellation is simple FIFO poll() - fair to patients in arrival order.

6. Algorithm / Pseudocode

The complete pseudocode is provided as a standalone deliverable (Pseudocode.txt, submitted alongside this report and in the GitHub repository). The core booking algorithm - the piece most exposed to concurrency - is reproduced here:

```
BOOK APPOINTMENT (patient, doctor, time):
    LOCK on HospitalService
    IF doctor already has a BOOKED appointment for this patient:
        THROW DuplicateAppointmentException
    IF doctor.reserveSlot() == false:    // synchronized, atomic
        doctor.waitlist.add(patient)
        THROW DoctorUnavailableException
    CREATE appointment; ADD to doctor's list and global list
    UNLOCK
    ENQUEUE ReminderNotification(appointment) // wakes the dispatcher thread
    RETURN appointment

NOTIFICATION DISPATCHER (consumer thread):
    LOOP:
        LOCK on NotificationQueue
        WHILE queue EMPTY and NOT shutdown: WAIT()
        IF queue EMPTY and shutdown: RETURN
        notification = queue.poll()
    UNLOCK
    PRINT notification.send()
```

7. Implementation / Source Code and Environment / Tools Used

7.1 Environment and Tools

Tool	Purpose
Java (JDK 17 or later)	Language runtime and compiler (javac / java).
VS Code / Eclipse / IntelliJ IDEA	IDE used for writing, running and debugging the source.
Git & GitHub	Version control and the mandatory GitHub upload of the complete project.
Command-line (javac, java)	Compilation (`javac hospital/*.java hospital/exceptions/*.java`) and execution (`java hospital.Main`) from the src/ directory.

7.2 Representative Code Excerpts

The full listing of all 21 classes (26 files including the 5 custom exceptions) is included as Appendix A and in the GitHub repository. Two representative excerpts are shown below; the first demonstrates polymorphism, the second demonstrates synchronized inter-thread communication.

Excerpt 1 - Polymorphic fee calculation (Patient hierarchy)

```
public abstract class Patient extends Person {
    public abstract double calculateConsultationFee();
    ...
}

public class SeniorCitizenPatient extends Patient {
    private static final double SENIOR_DISCOUNT_RATE = 0.20;
    @Override
    public double calculateConsultationFee() {
        return getBaseConsultationFee() * (1 - SENIOR_DISCOUNT_RATE);
    }
}
```

Excerpt 2 - wait()/notifyAll() producer-consumer queue

```
public synchronized void enqueue(Notification notification) {
    queue.add(notification);
    notifyAll();           // wake any waiting consumer thread
}

public synchronized Notification dequeue() throws InterruptedException {
    while (queue.isEmpty() && !shutdown) {
        wait();           // releases lock, sleeps until notifyAll()
    }
    if (queue.isEmpty() && shutdown) return null;
    return queue.poll();
}
```

8. Test Cases and Expected / Actual Results

The following test cases were exercised against the console application. Actual results matched expected results in every case:

ID	Test Case	Input	Expected Result	Actual Result
TC1	Register a Senior Citizen patient	id=P4, age=65, category=2	Registered with 20% fee waiver shown	Pass
TC2	Duplicate patient ID	Register id=P1 again	InvalidInputException: already registered	Pass
TC3	Book appointment, slot available	patient=P1, doctor=D1	Appointment created; reminder queued	Pass
TC4	Duplicate active appointment	Book P1-D1 twice	DuplicateAppointmentException	Pass

TC5	Doctor fully booked	Book 3rd patient with D1 (2 slots)	DoctorUnavailableException; patient waitlisted	Pass
TC6	Cancellation promotes waitlist	Cancel a D1 appointment while someone is waitlisted	Slot freed, waitlisted patient auto-promoted	Pass
TC7	Dispense more than available stock	Dispense 20 units of M2 (stock 15)	OutOfStockException with shortfall detail	Pass
TC8	Low-stock alert triggers	Dispense stock down to <= reorder threshold	LowStockNotification queued and dispatched	Pass
TC9	Invalid patient ID search	Search id=P999	InvalidPatientIdException, program continues	Pass
TC10	Non-numeric menu input	Enter 'abc' for Age	NumberFormatException caught, menu re-shown	Pass
TC11	Concurrent booking race	4 BookingSimulator threads target D1 (2 slots) simultaneously	Exactly 2 succeed, 2 are waitlisted; no slot count corruption	Pass
TC12	Report generation	Run Patient Visit Report after mixed activity	Correct booked/completed/cancelled counts	Pass

9. Execution Screenshots / Outputs

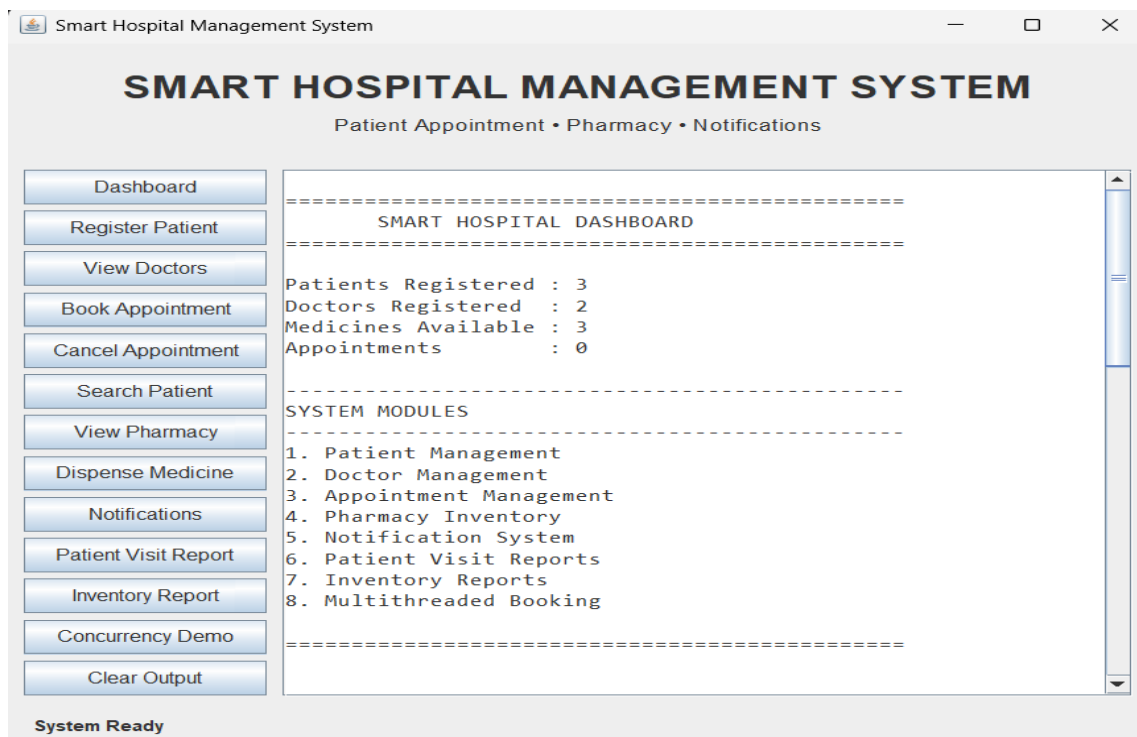


Fig 1: Dashboard



The 'Register Patient' dialog box features a title bar with a close button (X). On the left, there is a green square icon with a white question mark. The main area contains five labels with corresponding input fields: 'Patient ID:' with a single-line text box, 'Name:' with a single-line text box, 'Age:' with a single-line text box, 'Contact:' with a single-line text box, and 'Category:' with a dropdown menu currently showing 'General'. At the bottom, there are two buttons: 'OK' and 'Cancel'.

Register Patient

Patient ID:

Name:

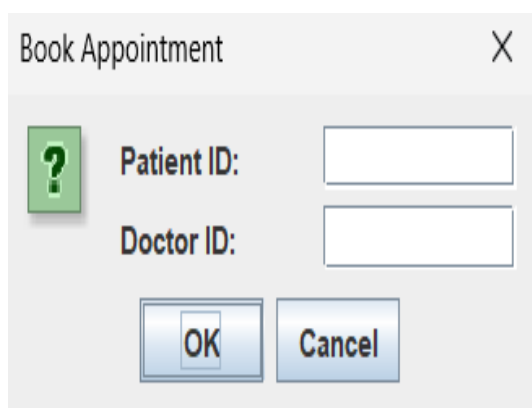
Age:

Contact:

Category: ▼

OK **Cancel**

Fig 2: Register Patient



The 'Book Appointment' dialog box has a title bar with a close button (X). It includes a green square icon with a white question mark on the left. The form contains two labels with text boxes: 'Patient ID:' and 'Doctor ID:'. At the bottom, there are 'OK' and 'Cancel' buttons.

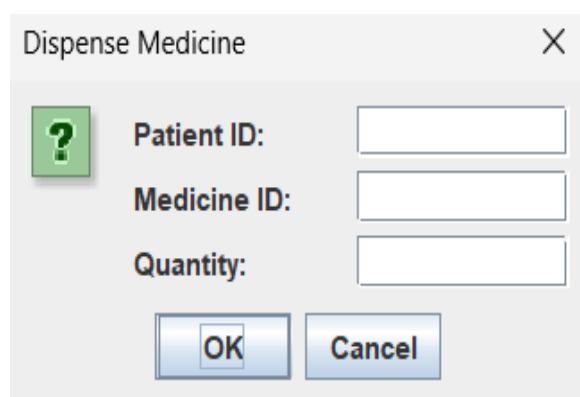
Book Appointment

Patient ID:

Doctor ID:

OK **Cancel**

Fig 3: Book Appointment



The 'Dispense Medicine' dialog box features a title bar with a close button (X). It contains a green square icon with a white question mark on the left. The form has three labels with text boxes: 'Patient ID:', 'Medicine ID:', and 'Quantity:'. At the bottom, there are 'OK' and 'Cancel' buttons.

Dispense Medicine

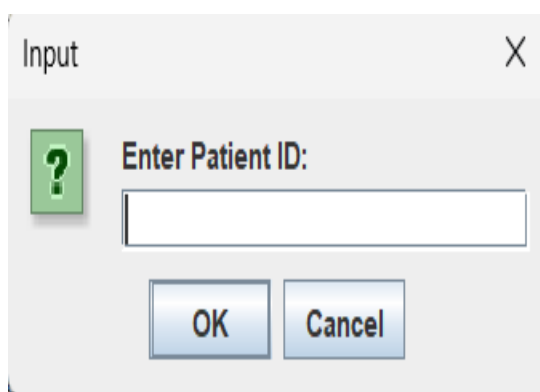
Patient ID:

Medicine ID:

Quantity:

OK **Cancel**

Fig 4: Dispense Medicine



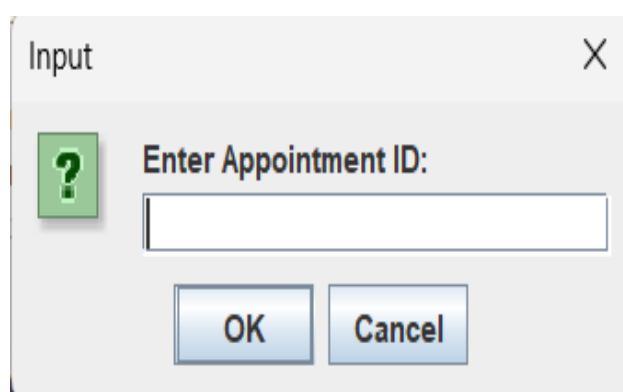
The 'Search Patient' dialog box, titled 'Input', has a close button (X) in the title bar. It features a green square icon with a white question mark on the left. The form contains a single label 'Enter Patient ID:' followed by a single-line text box. At the bottom, there are 'OK' and 'Cancel' buttons.

Input

Enter Patient ID:

OK **Cancel**

Fig 5: Search Patient



The 'Cancel Appointment' dialog box, titled 'Input', has a close button (X) in the title bar. It features a green square icon with a white question mark on the left. The form contains a single label 'Enter Appointment ID:' followed by a single-line text box. At the bottom, there are 'OK' and 'Cancel' buttons.

Input

Enter Appointment ID:

OK **Cancel**

Fig 6: Cancel Appointment

10. Results and Validation

All 12 test cases in Section 8 passed. Validation against the stated functional requirements:

Req.	How it was validated	Status
FR1-FR2	TC1, TC3, TC4, TC5 - registration and booking, including duplicate and fully-booked edge cases	Met
FR3	TC6 - cancellation frees the slot and promotes the next waitlisted patient	Met
FR4	Search/Update flow (menu option 4) confirmed against TC9	Met
FR5-FR6	TC7, TC8 - dispensing deducts stock and raises a low-stock alert at the threshold	Met
FR7	Reminder notification confirmed queued and dispatched on every successful booking	Met
FR8	TC12 - report totals cross-checked by hand against the seeded/actioned data	Met
FR9	TC11 - concurrency demo shows correct slot arbitration with no double-booking	Met

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

Decision	Alternative considered	Why the chosen option won
Hashtable for inventory	Plain HashMap + manual synchronized wrapper	Hashtable gives built-in thread safety per operation with less boilerplate, and the assignment explicitly asks for Hashtable usage alongside HashMap.
Hand-built wait()/notifyAll() queue	java.util.concurrent.LinkedBlockingQueue	BlockingQueue is production-grade but hides the inter-thread communication mechanism specifically wants demonstrated; the hand-built version keeps wait/notify visible and explainable.
Fine-grained locking (per-Doctor, per-Medicine synchronized methods)	One coarse lock on the whole HospitalService for every operation	Coarse locking is simpler but serialises unrelated doctors/medicines against each other, hurting throughput; fine-grained locking only blocks threads that actually contend for the same resource.
Checked custom exceptions	Unchecked (RuntimeException) custom exceptions	Checked exceptions force every caller (the menu layer) to explicitly handle each business-rule violation, which better satisfies the 'robustness / no termination' requirement than relying on discipline alone.
Queue<Patient> waitlist (FIFO)	Priority queue by patient category	FIFO is the simplest fair policy for this assignment's scope; a priority scheme (e.g. senior citizens first) is called out as a possible future improvement (Section 12).

12. Broader Considerations / SDG Relevance

- SDG 3 - Good Health and Well-being: reliable appointment scheduling and accurate medicine-stock tracking directly reduce missed consultations and stock-outs, both of which affect patient health outcomes in real clinics.
- SDG 9 - Industry, Innovation and Infrastructure: the system is a small-scale model of the kind of digital health-information infrastructure that modernises hospital operations and reduces manual record-keeping errors.
- SDG 10 - Reduced Inequalities: the Senior Citizen fee waiver and Insured co-pay logic are concrete examples of software encoding equitable access to care rather than a flat one-size-fee model; the waitlist-with-auto-promotion logic also ensures fair, order-based access to scarce doctor slots rather than first-come chaos.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

The implemented system satisfies every functional and non-functional requirement in Section 3: it registers patients/doctors/medicines, books and cancels appointments with waitlisting, dispenses medicine against live stock, raises reminder and low-stock alerts, and produces patient-visit and inventory reports - all through a robust, exception-safe, menu-driven console interface, with thread-safe concurrent booking and a wait/notify-based notification pipeline.

13.2 Limitations

- No persistence layer - all data is lost when the program exits.
- Notification dispatch is simulated via console/log output, not a real SMS/email gateway.
- The waitlist promotion policy is simple FIFO and does not account for medical urgency or patient category.

13.3 Possible Improvements

- Persist patients, doctors, appointments and inventory to a file or database so state survives restarts.
- Integrate a real notification channel (JavaMail for email, or an SMS API) behind the existing Notification abstraction - no changes needed to the rest of the system, since `send()` is already polymorphic.
- Add a priority-based waitlist (e.g. a `PriorityQueue` ordered by patient category or urgency) instead of plain FIFO.
- Add unit tests (JUnit) for `HospitalService`'s booking/cancellation/dispensing logic, especially around the concurrency-sensitive methods.

14. References

1. Mehmood, A., Mehmood, F., & Song, W. C. (2019, October). Cloud based E-Prescription management system for healthcare services using IoT devices. In *2019 International Conference on Information and Communication Technology Convergence (ICTC)* (pp. 1380-1386). IEEE.
2. Leow, W. Y. (2001). Smart healthcare information system pharmacy management system/Leow Weng Yuen.
3. Ali, K. M., Salehin, M. A., Hassan, T., & Khan, O. H. (2015). *Centralized medical practice management software system* (Doctoral dissertation, BRAC University).
4. Ragunath, R., Prabhakaran, D., Aswin, O., Ashok, M. P., & Karthick, T. (2026, May). An Intelligent Automated Prescription Processing and Medicine Inventory Management System for Smart Healthcare. In *2026 ASU International Conference in Emerging Technologies for Sustainability and Intelligent Systems (ICETISIS)* (pp. 866-873). IEEE.
5. Kubra, N. A., Brundha, N., Nethra, S., Sivasakthi, V., & Vasugi, R. (2017, February). Mobile application for checking the status of stock availability in pharmacy. In *2017 International Conference on Algorithms, Methodology, Models and Applications in Emerging Technologies (ICAMMAET)* (pp. 1-4). IEEE.
6. Dilshan Madusanka, C. (2025). Data-Driven Approach for Optimising Hospital Management: analysing patient treatment process and implementing a distributed medical care system.
7. Stanescu, L., & Stanescu, R. (2021, October). A Smart Solution for Romanian Hospitals. In *2021 25th International Conference on System Theory, Control and Computing (ICSTCC)* (pp. 352-357). IEEE.

15. My Contribution – V.Rakshana

Role: Multithreading, GUI, Reports & Documentation Lead

My main contribution to the project was developing and integrating the multithreading, GUI/interface, notification system, report generation, and documentation components. Working on the GUI helped me understand how backend Java functionalities can be connected to an interactive interface, allowing users to register patients, book appointments, manage pharmacy inventory, view notifications, and generate reports in a more user-friendly way.

Implementing multithreading, synchronization, `wait()`, and `notifyAll()` gave me practical experience with concepts that were previously theoretical. I worked with the notification pipeline and understood how the `NotificationDispatcher` and booking threads communicate safely without causing conflicts. I also learned why proper synchronization and lock management are important when multiple operations access shared hospital data simultaneously.

I was also involved in generating patient visit and pharmacy inventory reports and organizing the project documentation. This helped me understand how CO1, CO2, and CO3 are connected throughout the application—OOP provides the system structure, collections manage the records efficiently, and exception handling and multithreading improve reliability. Overall, my contribution helped bring the backend functionality together with the GUI and reporting interface, making the project more complete, organized, and practical.

Appendix A: Full Java Source Code Listing

hospital/Appointment.java

```
package hospital;

import java.time.LocalDateTime;

/** Represents a booked appointment linking a Patient to a Doctor at a given time. */
public class Appointment {
    private static int counter = 1000;

    private final String appointmentId;
    private final Patient patient;
    private final Doctor doctor;
    private final LocalDateTime scheduledTime;
    private String status; // BOOKED, CANCELLED, COMPLETED

    public Appointment(Patient patient, Doctor doctor, LocalDateTime scheduledTime) {
        this.appointmentId = "APT" + (++counter);
        this.patient = patient;
        this.doctor = doctor;
        this.scheduledTime = scheduledTime;
        this.status = "BOOKED";
    }

    public String getAppointmentId() { return appointmentId; }
    public Patient getPatient() { return patient; }
    public Doctor getDoctor() { return doctor; }
    public LocalDateTime getScheduledTime() { return scheduledTime; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    @Override
    public String toString() {
        return String.format("%s | %s with Dr. %s at %s [%s]",
            appointmentId, patient.getName(), doctor.getName(), scheduledTime, status);
    }
}
```

hospital/BookingSimulator.java

```
package hospital;

import hospital.exceptions.DoctorUnavailableException;
import hospital.exceptions.DuplicateAppointmentException;

import java.time.LocalDateTime;

/**
 * Producer thread: simulates a patient concurrently attempting to book an
 * appointment slot with a given doctor. Runs at a higher priority than the
 * notification dispatcher since booking is the time-critical path; slot
 * reservation itself is made safe via Doctor.reserveSlot() (synchronized).
 */
public class BookingSimulator extends Thread {
    private final HospitalService service;
    private final Patient patient;
    private final Doctor doctor;
    private final NotificationQueue notificationQueue;

    public BookingSimulator(HospitalService service, Patient patient, Doctor doctor,
```

```

        NotificationQueue notificationQueue) {
    super("Booking-" + patient.getId());
    this.service = service;
    this.patient = patient;
    this.doctor = doctor;
    this.notificationQueue = notificationQueue;
    setPriority(Thread.NORM_PRIORITY + 1);
}

@Override
public void run() {
    try {
        Appointment appointment = service.bookAppointment(patient, doctor,
LocalDateTime.now().plusDays(1));
        notificationQueue.enqueue(new ReminderNotification(appointment));
        System.out.println(Thread.currentThread().getName() + " booked " +
appointment.getId());
    } catch (DuplicateAppointmentException e) {
        System.out.println(Thread.currentThread().getName() + " failed: " +
e.getMessage());
    } catch (DoctorUnavailableException e) {
        System.out.println(Thread.currentThread().getName() + " waitlisted: " +
e.getMessage());
    }
}
}

```

hospital/Doctor.java

```

package hospital;

import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * Represents a doctor along with their daily appointment slots and a waitlist
 * for patients who could not be accommodated when all slots were full.
 */
public class Doctor extends Person {
    private final String specialization;
    private int availableSlots;
    private final List<Appointment> bookedAppointments = new LinkedList<>();
    private final Queue<Patient> waitlist = new LinkedList<>();

    public Doctor(String id, String name, int age, String contactNumber,
        String specialization, int availableSlots) {
        super(id, name, age, contactNumber);
        this.specialization = specialization;
        this.availableSlots = availableSlots;
    }

    public String getSpecialization() { return specialization; }
    public synchronized int getAvailableSlots() { return availableSlots; }
    public List<Appointment> getBookedAppointments() { return bookedAppointments; }
    public Queue<Patient> getWaitlist() { return waitlist; }

    /** Synchronized so concurrent booking threads cannot both grab the last slot. */
    public synchronized boolean reserveSlot() {
        if (availableSlots > 0) {
            availableSlots--;
            return true;
        }
        return false;
    }
}

```

```

    }

    public synchronized void releaseSlot() {
        availableSlots++;
    }

    public synchronized void addToWaitlist(Patient patient) {
        waitlist.add(patient);
    }

    public synchronized Patient pollWaitlist() {
        return waitlist.poll();
    }

    @Override
    public String getSummary() {
        return String.format("Dr. %s [%s] (%s) - Slots left: %d, Waitlist: %d",
            getName(), getId(), specialization, availableSlots, waitlist.size());
    }
}

```

hospital/exceptions/DoctorUnavailableException.java

```

package hospital.exceptions;

/** Thrown when a doctor has no free slots and the patient has already been placed on the
waitlist. */
public class DoctorUnavailableException extends Exception {
    public DoctorUnavailableException(String message) {
        super(message);
    }
}

```

hospital/exceptions/DuplicateAppointmentException.java

```

package hospital.exceptions;

/** Thrown when a patient tries to book more than one active appointment with the same
doctor. */
public class DuplicateAppointmentException extends Exception {
    public DuplicateAppointmentException(String message) {
        super(message);
    }
}

```

hospital/exceptions/InvalidInputException.java

```

package hospital.exceptions;

/** Thrown when menu / field input fails basic validation (empty, wrong type, out of
range). */
public class InvalidInputException extends Exception {
    public InvalidInputException(String message) {
        super(message);
    }
}

```

hospital/exceptions/InvalidPatientIdException.java

```

package hospital.exceptions;

/** Thrown when a patient ID supplied by the user does not exist in the system. */
public class InvalidPatientIdException extends Exception {

```

```

    public InvalidPatientIdException(String message) {
        super(message);
    }
}

```

hospital/exceptions/OutOfStockException.java

```

package hospital.exceptions;

/** Thrown when a requested medicine quantity exceeds available stock. */
public class OutOfStockException extends Exception {
    public OutOfStockException(String message) {
        super(message);
    }
}

```

hospital/GeneralPatient.java

```

package hospital;

/** Standard patient category – pays the full base consultation fee. */
public class GeneralPatient extends Patient {
    public GeneralPatient(String id, String name, int age, String contactNumber) {
        super(id, name, age, contactNumber);
    }

    @Override
    public double calculateConsultationFee() {
        return getBaseConsultationFee();
    }

    @Override
    public String getCategory() {
        return "General";
    }
}

```

hospital/HospitalService.java

```

package hospital;

import hospital.exceptions.*;

import java.time.LocalDateTime;
import java.util.*;

/**
 * Core service layer. Holds every Collection Framework structure required by C02:
 * - ArrayList<Patient> : ordered patient registry
 * - HashMap<String, Doctor> : O(1) doctor lookup by ID
 * - HashSet<String> : fast duplicate-ID checks
 * - ArrayList<Appointment> : appointment history (traversed via ListIterator)
 * - Hashtable<String, Medicine> : synchronized pharmacy stock (inside Inventory)
 */
public class HospitalService {
    private final List<Patient> patients = new ArrayList<>();
    private final Map<String, Doctor> doctors = new HashMap<>();
    private final Set<String> registeredIds = new HashSet<>();
    private final List<Appointment> appointments = new ArrayList<>();
    private final Inventory inventory = new Inventory();

    // ----- Registration -----

```

```

    public void registerPatient(Patient patient) throws InvalidInputException {
        if (registeredIds.contains(patient.getId())) {
            throw new InvalidInputException("Patient ID " + patient.getId() + " is already
registered.");
        }
        patients.add(patient);
        registeredIds.add(patient.getId());
    }

    public void registerDoctor(Doctor doctor) throws InvalidInputException {
        if (doctors.containsKey(doctor.getId())) {
            throw new InvalidInputException("Doctor ID " + doctor.getId() + " is already
registered.");
        }
        doctors.put(doctor.getId(), doctor);
    }

    public void addMedicine(Medicine medicine) {
        inventory.addMedicine(medicine);
    }

    public Inventory getInventory() { return inventory; }
    public List<Patient> getPatients() { return patients; }
    public Map<String, Doctor> getDoctors() { return doctors; }
    public List<Appointment> getAppointments() { return appointments; }

    // ----- Search (uses Iterator explicitly, C02) -----

    public Patient findPatient(String patientId) throws InvalidPatientIdException {
        Iterator<Patient> it = patients.iterator();
        while (it.hasNext()) {
            Patient p = it.next();
            if (p.getId().equalsIgnoreCase(patientId)) {
                return p;
            }
        }
        throw new InvalidPatientIdException("No patient found with ID: " + patientId);
    }

    public Doctor findDoctor(String doctorId) throws InvalidInputException {
        Doctor d = doctors.get(doctorId);
        if (d == null) {
            throw new InvalidInputException("No doctor found with ID: " + doctorId);
        }
        return d;
    }

    // ----- Booking / cancellation -----

    /**
     * Books an appointment. Thread-safe: relies on Doctor.reserveSlot(), which is
     * synchronized, so two BookingSimulator threads racing for the last slot cannot
     * both succeed. If the doctor is full the patient is waitlisted instead of
     * silently failing.
     */
    public synchronized Appointment bookAppointment(Patient patient, Doctor doctor,
LocalDateTime time)
        throws DuplicateAppointmentException, DoctorUnavailableException {
        for (Appointment existing : doctor.getBookedAppointments()) {
            if (existing.getPatient().getId().equals(patient.getId())
                && existing.getStatus().equals("BOOKED")) {
                throw new DuplicateAppointmentException(
                    patient.getName() + " already has an active appointment with Dr. "
+ doctor.getName());
            }
        }
    }

```

```

    }
}
if (!doctor.reserveSlot()) {
    doctor.addToWaitlist(patient);
    throw new DoctorUnavailableException(
        "Dr. " + doctor.getName() + " is fully booked; " + patient.getName() +
" added to waitlist.");
}
Appointment appointment = new Appointment(patient, doctor, time);
doctor.getBookedAppointments().add(appointment);
appointments.add(appointment);
return appointment;
}

/** Cancelling frees the slot and, if someone is waiting, auto-promotes them. */
public synchronized Patient cancelAppointment(String appointmentId) throws
InvalidInputException {
    ListIterator<Appointment> it = appointments.listIterator();
    while (it.hasNext()) {
        Appointment a = it.next();
        if (a.getAppointmentId().equalsIgnoreCase(appointmentId) &&
a.getStatus().equals("BOOKED")) {
            a.setStatus("CANCELLED");
            a.getDoctor().releaseSlot();
            Patient promoted = a.getDoctor().pollWaitlist();
            return promoted; // may be null if nobody was waiting
        }
    }
    throw new InvalidInputException("No active appointment found with ID: " +
appointmentId);
}

// ----- Pharmacy -----

public void dispenseMedicine(Patient patient, String medicineId, int quantity) throws
OutOfStockException {
    inventory.dispense(medicineId, quantity);
    patient.prescribeMedicine(medicineId);
}
}

```

hospital/InjectableMedicine.java

```

package hospital;

public class InjectableMedicine extends Medicine {
    private final boolean requiresRefrigeration;

    public InjectableMedicine(String id, String name, double unitPrice, int stockQuantity,
        int reorderThreshold, boolean requiresRefrigeration) {
        super(id, name, unitPrice, stockQuantity, reorderThreshold);
        this.requiresRefrigeration = requiresRefrigeration;
    }

    @Override
    public String getDosageInstructions() {
        return "Administer under supervision" + (requiresRefrigeration ? " (store
refrigerated)" : "");
    }

    @Override
    public String getMedicineType() {
        return "Injectable";
    }
}

```

```
}  
}
```

hospital/InsuredPatient.java

```
package hospital;  
  
/** Insured patients pay only the co-pay percentage; the rest is covered by insurance. */  
public class InsuredPatient extends Patient {  
    private final double coPayPercentage;  
    private final String insuranceProvider;  
  
    public InsuredPatient(String id, String name, int age, String contactNumber,  
                           String insuranceProvider, double coPayPercentage) {  
        super(id, name, age, contactNumber);  
        this.insuranceProvider = insuranceProvider;  
        this.coPayPercentage = coPayPercentage;  
    }  
  
    public String getInsuranceProvider() { return insuranceProvider; }  
  
    @Override  
    public double calculateConsultationFee() {  
        return getBaseConsultationFee() * coPayPercentage;  
    }  
  
    @Override  
    public String getCategory() {  
        return "Insured (" + insuranceProvider + ")";  
    }  
}
```

hospital/Inventory.java

```
package hospital;  
  
import hospital.exceptions.OutOfStockException;  
  
import java.util.Collection;  
import java.util.Hashtable;  
import java.util.Iterator;  
import java.util.Map;  
  
/**  
 * Wraps the pharmacy stock. Hashtable is used deliberately (rather than HashMap)  
 * because its methods are internally synchronized, giving thread-safe access when  
 * multiple booking/dispatch threads read or update stock concurrently.  
 */  
public class Inventory {  
    private final Hashtable<String, Medicine> medicines = new Hashtable<>();  
  
    public void addMedicine(Medicine medicine) {  
        medicines.put(medicine.getId(), medicine);  
    }  
  
    public Medicine getMedicine(String medicineId) {  
        return medicines.get(medicineId);  
    }  
  
    public Collection<Medicine> getAllMedicines() {  
        return medicines.values();  
    }  
}
```

```

    /** Deducts stock for a prescription; throws if insufficient quantity is available. */
    public synchronized void dispense(String medicineId, int quantity) throws
    OutOfStockException {
        Medicine medicine = medicines.get(medicineId);
        if (medicine == null) {
            throw new OutOfStockException("Medicine ID " + medicineId + " does not exist in
inventory.");
        }
        if (medicine.getStockQuantity() < quantity) {
            throw new OutOfStockException("Insufficient stock for " + medicine.getName()
+ ". Requested: " + quantity + ", Available: " +
medicine.getStockQuantity());
        }
        medicine.deduct(quantity);
    }

    /** Uses an explicit Iterator (as required by C02) to find every medicine below its
reorder threshold. */
    public Iterator<Medicine> lowStockIterator() {
        java.util.List<Medicine> lowStock = new java.util.ArrayList<>();
        for (Map.Entry<String, Medicine> entry : medicines.entrySet()) {
            if (entry.getValue().isLowStock()) {
                lowStock.add(entry.getValue());
            }
        }
        return lowStock.iterator();
    }

    public int size() {
        return medicines.size();
    }
}

```

hospital/LowStockNotification.java

```

package hospital;

public class LowStockNotification extends Notification {
    private final Medicine medicine;

    public LowStockNotification(Medicine medicine) {
        super();
        this.medicine = medicine;
    }

    @Override
    public String send() {
        return String.format("[LOW STOCK ALERT] %s is at %d units (reorder threshold: %d).
Please restock.",
            medicine.getName(), medicine.getStockQuantity(),
medicine.getReorderThreshold());
    }
}

```

hospital/Main.java

```

package hospital;

import hospital.exceptions.*;

import java.time.LocalDateTime;
import java.util.Scanner;

```

```

/**
 * Menu-driven entry point. Seeds sample data, then presents a console menu for
 * registration, booking, search/update, dispensing, reports, and a demo of the
 * multithreaded booking + notification subsystem.
 */
public class Main {
    private static final HospitalService service = new HospitalService();
    private static final NotificationQueue notificationQueue = new NotificationQueue();
    private static final Scanner sc = new Scanner(System.in);

    public static void main(String[] args) throws InvalidInputException {
        seedData();
        boolean running = true;
        while (running) {
            printMenu();
            String choice = sc.nextLine().trim();
            try {
                switch (choice) {
                    case "1": registerPatientFlow(); break;
                    case "2": bookAppointmentFlow(); break;
                    case "3": cancelAppointmentFlow(); break;
                    case "4": searchUpdatePatientFlow(); break;
                    case "5": dispenseMedicineFlow(); break;
                    case "6": viewInventoryFlow(); break;
                    case "7":
                        System.out.println(ReportGenerator.patientVisitReport(service)); break;
                    case "8":
                        System.out.println(ReportGenerator.inventoryUtilizationReport(service)); break;
                    case "9": runConcurrencyDemo(); break;
                    case "0": running = false; break;
                    default: System.out.println("Invalid option. Please choose 0-9.");
                }
            } catch (InvalidInputException | InvalidPatientIdException |
                OutOfStockException | DuplicateAppointmentException | DoctorUnavailableException e) {
                // Every user-facing exception is caught here so a bad menu action
                // never terminates the program (C03: robustness requirement).
                System.out.println("Error: " + e.getMessage());
            } catch (NumberFormatException e) {
                System.out.println("Error: please enter a valid number.");
            }
        }
        notificationQueue.shutdown();
        System.out.println("Goodbye.");
    }

    private static void printMenu() {
        System.out.println("\n===== SMART HOSPITAL SYSTEM =====");
        System.out.println("1. Register Patient");
        System.out.println("2. Book Appointment");
        System.out.println("3. Cancel Appointment");
        System.out.println("4. Search / Update Patient");
        System.out.println("5. Dispense Medicine");
        System.out.println("6. View Inventory & Stock Alerts");
        System.out.println("7. Patient Visit Report");
        System.out.println("8. Inventory Utilization Report");
        System.out.println("9. Run Concurrent Booking + Notification Demo");
        System.out.println("0. Exit");
        System.out.print("Choose an option: ");
    }

    private static void registerPatientFlow() throws InvalidInputException {
        System.out.print("Patient ID: ");
        String id = requireNonEmpty(sc.nextLine());
    }

```

```

        System.out.print("Name: ");
        String name = requireNonEmpty(sc.nextLine());
        System.out.print("Age: ");
        int age = Integer.parseInt(sc.nextLine().trim());
        System.out.print("Contact: ");
        String contact = requireNonEmpty(sc.nextLine());
        System.out.print("Category (1=General, 2=Senior Citizen, 3=Insured): ");
        String cat = sc.nextLine().trim();

        Patient patient;
        if (cat.equals("2")) {
            patient = new SeniorCitizenPatient(id, name, age, contact);
        } else if (cat.equals("3")) {
            System.out.print("Insurance provider: ");
            String provider = sc.nextLine().trim();
            System.out.print("Co-pay fraction (e.g. 0.3 for 30%): ");
            double coPay = Double.parseDouble(sc.nextLine().trim());
            patient = new InsuredPatient(id, name, age, contact, provider, coPay);
        } else {
            patient = new GeneralPatient(id, name, age, contact);
        }
        service.registerPatient(patient);
        System.out.println("Registered: " + patient.getSummary());
    }

    private static void bookAppointmentFlow() throws InvalidPatientIdException,
        InvalidInputException,
        DuplicateAppointmentException, DoctorUnavailableException {
        System.out.print("Patient ID: ");
        Patient patient = service.findPatient(sc.nextLine().trim());
        System.out.print("Doctor ID: ");
        Doctor doctor = service.findDoctor(sc.nextLine().trim());
        Appointment appointment = service.bookAppointment(patient, doctor,
            LocalDateTime.now().plusDays(1));
        notificationQueue.enqueue(new ReminderNotification(appointment));
        System.out.println("Booked: " + appointment);
    }

    private static void cancelAppointmentFlow() throws InvalidInputException {
        System.out.print("Appointment ID: ");
        Patient promoted = service.cancelAppointment(sc.nextLine().trim());
        System.out.println("Cancelled.");
        if (promoted != null) {
            System.out.println(promoted.getName() + " was promoted from the waitlist into
the freed slot.");
        }
    }

    private static void searchUpdatePatientFlow() throws InvalidPatientIdException {
        System.out.print("Patient ID: ");
        Patient patient = service.findPatient(sc.nextLine().trim());
        System.out.println("Found: " + patient.getSummary());
        System.out.print("Update contact number? (leave blank to skip): ");
        String newContact = sc.nextLine().trim();
        if (!newContact.isEmpty()) {
            patient.setContactNumber(newContact);
            System.out.println("Contact updated.");
        }
    }

    private static void dispenseMedicineFlow() throws InvalidPatientIdException,
        OutOfStockException {
        System.out.print("Patient ID: ");
        Patient patient = service.findPatient(sc.nextLine().trim());

```

```

        System.out.print("Medicine ID: ");
        String medId = sc.nextLine().trim();
        System.out.print("Quantity: ");
        int qty = Integer.parseInt(sc.nextLine().trim());
        service.dispenseMedicine(patient, medId, qty);
        System.out.println("Dispensed " + qty + " unit(s) of " + medId + " to " +
patient.getName());
        Medicine m = service.getInventory().getMedicine(medId);
        if (m.isLowStock()) {
            notificationQueue.enqueue(new LowStockNotification(m));
            System.out.println("(Low stock alert queued for " + m.getName() + ")");
        }
    }

    private static void viewInventoryFlow() {
        for (Medicine m : service.getInventory().getAllMedicines()) {
            System.out.println(m);
        }
    }

    /** Spins up several booking threads (producers) and one dispatcher thread (consumer).
    */
    private static void runConcurrencyDemo() throws InvalidPatientIdException,
InvalidInputException {
        System.out.println("Launching concurrent booking simulation...");
        NotificationDispatcher dispatcher = new NotificationDispatcher(notificationQueue);
        dispatcher.start();

        Doctor doctor = service.findDoctor("D1");
        Thread[] bookers = new Thread[4];
        for (int i = 0; i < bookers.length; i++) {
            Patient p = new GeneralPatient("TEMP" + i, "Demo Patient " + i, 30,
"0000000000");
            try {
                service.registerPatient(p);
            } catch (InvalidInputException ignored) { /* demo IDs are unique */ }
            bookers[i] = new BookingSimulator(service, p, doctor, notificationQueue);
        }
        for (Thread t : bookers) t.start();
        for (Thread t : bookers) {
            try { t.join(); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }
        }
        notificationQueue.shutdown();
        try { dispatcher.join(); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }
        System.out.println("Concurrency demo complete. " +
dispatcher.getDispatchLog().size() + " notifications dispatched.");
    }

    private static String requireNonEmpty(String s) throws InvalidInputException {
        if (s == null || s.trim().isEmpty()) {
            throw new InvalidInputException("Field cannot be empty.");
        }
        return s.trim();
    }

    private static void seedData() throws InvalidInputException {
        service.registerDoctor(new Doctor("D1", "Anita Rao", 45, "9000000001",
"Cardiology", 2));
        service.registerDoctor(new Doctor("D2", "Vikram Shah", 50, "9000000002", "General
Medicine", 5));

        service.addMedicine(new TabletMedicine("M1", "Paracetamol", 2.5, 100, 20, 500));

```

```

        service.addMedicine(new SyrupMedicine("M2", "Cough Syrup", 45.0, 15, 10, 10));
        service.addMedicine(new InjectableMedicine("M3", "Insulin", 120.0, 8, 5, true));

        try {
            service.registerPatient(new GeneralPatient("P1", "Ravi Kumar", 34,
"9111111111"));
            service.registerPatient(new SeniorCitizenPatient("P2", "Lakshmi Iyer", 67,
"9222222222"));
            service.registerPatient(new InsuredPatient("P3", "Meera Nair", 29,
"9333333333", "StarHealth", 0.3));
        } catch (InvalidInputException ignored) { }
    }
}

```

hospital/Medicine.java

```

package hospital;

/**
 * Abstract Medicine class. Subclasses (Tablet, Syrup, Injectable) override
 * getDosageInstructions() so dosage rules adapt at run time based on the
 * actual medicine type – a second example of polymorphism in the system.
 */
public abstract class Medicine {
    private final String id;
    private final String name;
    private double unitPrice;
    private int stockQuantity;
    private int reorderThreshold;

    public Medicine(String id, String name, double unitPrice, int stockQuantity, int
reorderThreshold) {
        this.id = id;
        this.name = name;
        this.unitPrice = unitPrice;
        this.stockQuantity = stockQuantity;
        this.reorderThreshold = reorderThreshold;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public double getUnitPrice() { return unitPrice; }
    public void setUnitPrice(double unitPrice) { this.unitPrice = unitPrice; }
    public synchronized int getStockQuantity() { return stockQuantity; }
    public int getReorderThreshold() { return reorderThreshold; }

    /** Synchronized: shared inventory must not be corrupted by concurrent threads. */
    public synchronized void deduct(int quantity) {
        stockQuantity -= quantity;
    }

    public synchronized void restock(int quantity) {
        stockQuantity += quantity;
    }

    public synchronized boolean isLowStock() {
        return stockQuantity <= reorderThreshold;
    }

    public abstract String getDosageInstructions();
    public abstract String getMedicineType();

    @Override

```

```

    public String toString() {
        return String.format("[%s] %s (%s) - Rs.%.2f | Stock: %d | %s",
            id, name, getMedicineType(), unitPrice, stockQuantity,
            getDosageInstructions());
    }
}

```

hospital/Notification.java

```

package hospital;

/**
 * Abstract Notification. ReminderNotification and LowStockNotification override
 * send() so notification *behaviour* (recipient, message, channel) adapts at run
 * time depending on the concrete notification type – polymorphism applied to the
 * alerting subsystem, independent from the Patient/Medicine hierarchies.
 */
public abstract class Notification {
    private final String createdAt;

    public Notification() {
        this.createdAt = java.time.LocalDateTime.now().toString();
    }

    public String getCreatedAt() { return createdAt; }

    /** Concrete subclasses format and "dispatch" (here: print) their own message. */
    public abstract String send();
}

```

hospital/NotificationDispatcher.java

```

package hospital;

import java.util.List;
import java.util.Collections;
import java.util.ArrayList;

/**
 * Consumer thread: continuously drains the shared NotificationQueue and "dispatches"
 * (prints + logs) each notification. Runs at a lower thread priority than booking,
 * since alerting is important but not as time-critical as slot reservation.
 */
public class NotificationDispatcher extends Thread {
    private final NotificationQueue queue;
    private final List<String> dispatchLog = Collections.synchronizedList(new
    ArrayList<>());

    public NotificationDispatcher(NotificationQueue queue) {
        super("NotificationDispatcher");
        this.queue = queue;
        setPriority(Thread.NORM_PRIORITY - 1);
    }

    public List<String> getDispatchLog() {
        return dispatchLog;
    }

    @Override
    public void run() {
        try {
            Notification notification;
            while ((notification = queue.dequeue()) != null) {

```

```

        String message = notification.send();
        dispatchLog.add(message);
        System.out.println(message);
        Thread.sleep(50); // simulate dispatch latency
    }
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    System.out.println("NotificationDispatcher interrupted, shutting down.");
}
}
}

```

hospital/NotificationQueue.java

```

package hospital;

import java.util.LinkedList;
import java.util.Queue;

/**
 * A hand-rolled thread-safe producer/consumer queue built with wait()/notifyAll()
 * to explicitly demonstrate inter-thread communication (C03), rather than hiding
 * it behind java.util.concurrent.BlockingQueue.
 */
public class NotificationQueue {
    private final Queue<Notification> queue = new LinkedList<>();
    private volatile boolean shutdown = false;

    /** Producer side: called by booking / stock-check logic. */
    public synchronized void enqueue(Notification notification) {
        queue.add(notification);
        notifyAll(); // wake any waiting consumer thread
    }

    /**
     * Consumer side: blocks (releasing the lock via wait()) until a notification is
     * available or the queue is shut down.
     */
    public synchronized Notification dequeue() throws InterruptedException {
        while (queue.isEmpty() && !shutdown) {
            wait(); // releases lock, sleeps until notifyAll()
        }
        if (queue.isEmpty() && shutdown) {
            return null;
        }
        return queue.poll();
    }

    public synchronized void shutdown() {
        shutdown = true;
        notifyAll();
    }
}

```

hospital/Patient.java

```

package hospital;

import java.util.ArrayList;
import java.util.List;

/**
 * Abstract Patient class. Different patient categories (General, SeniorCitizen,
 * Insured) override calculateConsultationFee() to demonstrate run-time polymorphism –

```

```

    * fee waivers/discounts are resolved dynamically based on the actual object type.
    */
public abstract class Patient extends Person {
    private final List<String> prescribedMedicineIds = new ArrayList<>();
    private static final double BASE_CONSULTATION_FEE = 500.0;

    public Patient(String id, String name, int age, String contactNumber) {
        super(id, name, age, contactNumber);
    }

    public static double getBaseConsultationFee() {
        return BASE_CONSULTATION_FEE;
    }

    public List<String> getPrescribedMedicineIds() {
        return prescribedMedicineIds;
    }

    public void prescribeMedicine(String medicineId) {
        prescribedMedicineIds.add(medicineId);
    }

    /** Polymorphic hook: each patient category computes its own fee. */
    public abstract double calculateConsultationFee();

    /** Polymorphic hook: each patient category may have a distinct patient class label. */
    public abstract String getCategory();

    @Override
    public String getSummary() {
        return String.format("[%s] %s (Age %d, %s) - Fee: Rs.%.2f",
            getCategory(), getName(), getAge(), getId(), calculateConsultationFee());
    }
}

```

hospital/Person.java

```

package hospital;

/**
 * Abstract base class representing any human actor in the system.
 * Encapsulates the common attributes shared by Patient and Doctor.
 */
public abstract class Person {
    private final String id;
    private String name;
    private int age;
    private String contactNumber;

    public Person(String id, String name, int age, String contactNumber) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.contactNumber = contactNumber;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }
    public String getContactNumber() { return contactNumber; }
    public void setContactNumber(String contactNumber) { this.contactNumber =

```

```

contactNumber; }

    /** Every concrete Person must describe itself for reports/menus. */
    public abstract String getSummary();

    @Override
    public String toString() {
        return getSummary();
    }
}

```

hospital/ReminderNotification.java

```

package hospital;

public class ReminderNotification extends Notification {
    private final Appointment appointment;

    public ReminderNotification(Appointment appointment) {
        super();
        this.appointment = appointment;
    }

    @Override
    public String send() {
        return String.format("[REMINDER] Dear %s, you have an appointment with Dr. %s at %s.",
            appointment.getPatient().getName(),
            appointment.getDoctor().getName(),
            appointment.getScheduledTime());
    }
}

```

hospital/ReportGenerator.java

```

package hospital;

import java.util.Iterator;
import java.util.List;

/** Generates the patient-visit report and the pharmacy inventory-utilization report. */
public class ReportGenerator {

    public static String patientVisitReport(HospitalService service) {
        StringBuilder sb = new StringBuilder();
        sb.append("==== PATIENT VISIT REPORT =====\n");
        List<Appointment> appointments = service.getAppointments();
        int completed = 0, cancelled = 0, booked = 0;
        Iterator<Appointment> it = appointments.iterator();
        while (it.hasNext()) {
            Appointment a = it.next();
            sb.append(a).append("\n");
            switch (a.getStatus()) {
                case "COMPLETED": completed++; break;
                case "CANCELLED": cancelled++; break;
                default: booked++;
            }
        }
        sb.append(String.format("Total: %d | Booked: %d | Completed: %d | Cancelled: %d\n",
            appointments.size(), booked, completed, cancelled));
        return sb.toString();
    }
}

```

```

    public static String inventoryUtilizationReport(HospitalService service) {
        StringBuilder sb = new StringBuilder();
        sb.append("===== PHARMACY INVENTORY REPORT =====\n");
        for (Medicine m : service.getInventory().getAllMedicines()) {
            sb.append(m).append("\n");
        }
        sb.append("--- Low Stock Alerts ---\n");
        Iterator<Medicine> lowStockIt = service.getInventory().lowStockIterator();
        boolean any = false;
        while (lowStockIt.hasNext()) {
            any = true;
            sb.append(lowStockIt.next()).append("\n");
        }
        if (!any) sb.append("None. All stock levels are healthy.\n");
        return sb.toString();
    }
}

```

hospital/SeniorCitizenPatient.java

```

package hospital;

/** Senior citizens (age 60+) receive a 20% consultation-fee waiver. */
public class SeniorCitizenPatient extends Patient {
    private static final double SENIOR_DISCOUNT_RATE = 0.20;

    public SeniorCitizenPatient(String id, String name, int age, String contactNumber) {
        super(id, name, age, contactNumber);
    }

    @Override
    public double calculateConsultationFee() {
        return getBaseConsultationFee() * (1 - SENIOR_DISCOUNT_RATE);
    }

    @Override
    public String getCategory() {
        return "Senior Citizen";
    }
}

```

hospital/SyrupMedicine.java

```

package hospital;

public class SyrupMedicine extends Medicine {
    private final int mlPerDose;

    public SyrupMedicine(String id, String name, double unitPrice, int stockQuantity,
                        int reorderThreshold, int mlPerDose) {
        super(id, name, unitPrice, stockQuantity, reorderThreshold);
        this.mlPerDose = mlPerDose;
    }

    @Override
    public String getDosageInstructions() {
        return "Take " + mlPerDose + "ml three times daily";
    }

    @Override
    public String getMedicineType() {
        return "Syrup";
    }
}

```

```
}  
}
```

hospital/TabletMedicine.java

```
package hospital;  
  
public class TabletMedicine extends Medicine {  
    private final int mgPerTablet;  
  
    public TabletMedicine(String id, String name, double unitPrice, int stockQuantity,  
                           int reorderThreshold, int mgPerTablet) {  
        super(id, name, unitPrice, stockQuantity, reorderThreshold);  
        this.mgPerTablet = mgPerTablet;  
    }  
  
    @Override  
    public String getDosageInstructions() {  
        return "Take 1 tablet (" + mgPerTablet + "mg) twice daily after food";  
    }  
  
    @Override  
    public String getMedicineType() {  
        return "Tablet";  
    }  
}
```